AI-Powered Task & Knowledge Management System

Technical Project Report

Python Full-Stack Development Assignment
FastAPI · React.js · MySQL · FAISS · JWT

Prepared for Technical Review
July 2026

Table of Contents

1. Introduction

Modern teams accumulate knowledge faster than they can organize it. Documents pile up in shared drives, and finding the right piece of information often means manually searching file names or asking a colleague. At the same time, day-to-day work still needs to be assigned, tracked, and closed out. This project addresses both problems in a single, minimal but production-shaped system: an Admin curates a knowledge base and assigns tasks, while Users search that knowledge base using AI-powered semantic search and complete the tasks assigned to them.

The system was built as a full-stack MVP within a 2-3 day scope, with the explicit goal of demonstrating system design ability rather than surface-level feature completeness — clean separation of concerns, a normalized relational schema, secure role-based access, and a semantic search pipeline implemented from first principles rather than delegated entirely to a third-party LLM API.

1.1 Objective

- Allow an Admin to build a searchable knowledge base by uploading documents.
- Allow Users to query that knowledge base using natural-language, meaning-based search rather than exact keyword matching.
- Allow Admins to create and assign tasks, and Users to track and update their status.
- Secure every API behind JWT authentication with role-based access control.
- Log key user activity and surface it through a basic analytics view.

2. Problem Statement

Design and implement a system where an Admin builds a knowledge base of documents, Users use AI-powered search to find answers within that knowledge base, and Users complete tasks assigned to them using the knowledge they retrieve. The system must be a working MVP, not a prototype with mocked-out core logic — in particular, the semantic search must be backed by a real embedding and vector-retrieval pipeline that the project implements directly.

3. Requirements

3.1 Functional Requirements

1. Authentication & RBAC: JWT-based login; Admin and User roles; every API protected according to role.
2. Relational database (MySQL) with a normalized schema covering users, roles, tasks, documents, and activity_logs, using primary and foreign keys.
3. Document upload (.txt, PDF optional) with metadata persisted for downstream AI processing.
4. Semantic search: text converted to embeddings, stored in a vector database (FAISS), queried by converting the search text into an embedding and retrieving nearest neighbours — implemented directly rather than delegated to an LLM API.
5. Task management: Admin creates and assigns tasks; User views tasks and updates status from Pending to Completed.
6. Dynamic filtering on at least one API, e.g. filtering tasks by status or assignee.
7. Activity logging for login, task updates, document uploads, and searches.
8. Basic analytics: total tasks, completed vs. pending, and most-searched queries.

3.2 Non-Functional Requirements

- Clean, structured, and scalable backend code with a clear separation between routing, business logic, and data access.
- A frontend with sensible component boundaries and centralized state/API handling.
- Security: password hashing, signed JWTs, and role checks enforced server-side, not just hidden in the UI.
- Maintainable folder structure and consistent naming conventions across both frontend and backend.

4. System Architecture

The system follows a layered client-server architecture. The React frontend never talks to the database or the vector index directly — every interaction goes through the FastAPI backend's REST API, authenticated with a JWT. The backend fans requests out to three functional modules (Authentication, Task Management, Document Management), and document text flows through a dedicated AI pipeline: an Embedding Service converts text into vectors, which are indexed in FAISS for similarity search. All structured, relational data lives in MySQL.

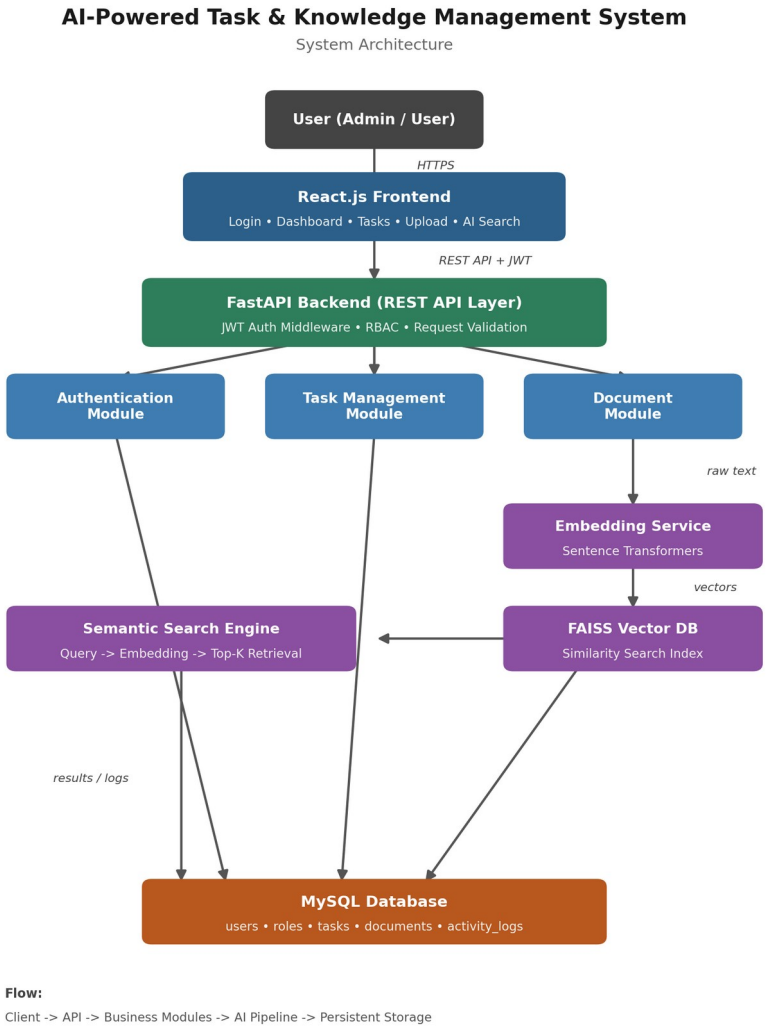

Figure 1: High-level system architecture — client, API layer, business modules, AI pipeline, and persistence.

4.1 Architectural Layers

- Presentation Layer — React.js frontend: login, dashboard, task board, document upload, and AI search UI.
- API Layer — FastAPI: request validation, JWT verification, and RBAC enforcement before any business logic runs.
- Business Logic Layer — dedicated service modules for authentication, tasks, and documents, kept separate from route handlers.
- AI Layer — Embedding Service (Sentence Transformers) and FAISS vector index, powering semantic search independently of any external LLM API.
- Persistence Layer — MySQL, storing users, roles, tasks, documents, and activity logs with enforced relationships.

5. Technology Stack & Justification

Layer	Technology	Why
Frontend	React.js	Component-based UI, large ecosystem, straightforward state management for a dashboard-style app.
Backend	FastAPI	Async-first, automatic OpenAPI/Swagger docs, strong typing via Pydantic — fast to build and easy to verify.
Database	MySQL	Mature relational engine; well suited to enforcing FK relationships across users, tasks, and documents.
ORM	SQLAlchemy	Explicit, well-documented mapping between Python models and relational tables.
Auth	JWT	Stateless, signed tokens that scale horizontally without server-side session storage.
Embeddings	Sentence Transformers	Local, open-source embedding generation — no dependency on a paid external API for core functionality.
Vector Search	FAISS	High-performance similarity search over dense vectors, built for exactly this retrieval problem.

6. Database Design

The schema is intentionally minimal but fully normalized, with every relationship expressed through explicit foreign keys rather than duplicated data.

Table	Key Columns	Relationships
roles	id (PK), name	Referenced by users.role_id
users	id (PK), name, email, password_hash, role_id (FK)	role_id → roles.id
tasks	id (PK), title, status, assigned_to (FK), created_by (FK)	assigned_to / created_by → users.id
documents	id (PK), filename, uploaded_by (FK),	uploaded_by → users.id

Table	Key Columns	Relationships
	uploaded_at	
activity_logs	id (PK), user_id (FK), action, details, timestamp	user_id → users.id

This structure keeps referential integrity at the database level: a task cannot be assigned to a non-existent user, a document cannot exist without a known uploader, and every logged action is traceable back to the user who triggered it.

7. Module-Wise Implementation

7.1 Authentication & RBAC

Users authenticate via /auth/login with email and password. Passwords are stored as salted hashes, never in plaintext. On successful login, the backend issues a signed JWT containing the user's ID and role. Every protected route depends on a shared FastAPI dependency that decodes and verifies this token, then checks the embedded role against the route's required permissions — so access control is enforced centrally rather than duplicated in every handler.

7.2 Document Upload & Knowledge Base

Admins upload .txt documents through the /documents endpoint. The raw file is stored, and its metadata (filename, uploader, timestamp) is written to the documents table. The document's text content is then handed to the embedding pipeline so it becomes searchable immediately after upload.

7.3 AI-Powered Semantic Search

This is the core differentiator of the system. Rather than sending user queries to an external LLM and returning whatever it says, the project implements the retrieval logic directly:

- Each document is chunked and passed through a Sentence Transformers model to produce a dense vector embedding capturing its semantic meaning.
- Embeddings are added to a FAISS index, which supports fast approximate nearest-neighbour search over high-dimensional vectors.
- When a user submits a query via /search, the query text is embedded using the same model, and FAISS returns the top-K most semantically similar documents — not just those containing matching keywords.
- Every search query is logged, feeding directly into the analytics view's 'most searched queries' metric.

This design satisfies the assignment's explicit constraint of not relying solely on an LLM API — the embedding generation and similarity search are core logic owned by the project.

7.4 Task Management

Admins create tasks and assign them to specific Users. Users see only the tasks relevant to them (or all tasks, if Admin) and can transition a task's status from Pending to Completed. Every creation and status change is timestamped and attributable to the acting user.

7.5 Dynamic Filtering

The /tasks endpoint accepts optional query parameters such as status and assigned_to, letting the frontend request exactly the slice of data it needs (e.g. /tasks?status=completed or /tasks?assigned_to=1) instead of fetching everything and filtering client-side.

7.6 Activity Logging

A lightweight logging service writes a row to activity_logs on four key events: login, document upload, search, and task update. Each entry captures the acting user, the action type, and a timestamp, forming an auditable trail of system usage.

7.7 Analytics

The /analytics endpoint aggregates data already captured elsewhere in the system: total task count, a completed-vs-pending breakdown, and a ranked list of the most frequent search queries pulled from the activity log — giving the Admin a quick operational overview without any separate reporting infrastructure.

8. API Design

Endpoint	Method	Description	Access
/auth/login	POST	Authenticate and issue a JWT	Public
/tasks	GET / POST	List (with filters), create tasks	Admin / User
/documents	GET / POST	Upload and list documents	Admin / User
/search	GET	Semantic search over the knowledge base	Admin / User
/analytics	GET	Task and search analytics	Admin

9. Application Workflow

9. User logs in; backend verifies credentials and issues a JWT.
10. Frontend stores the token and attaches it to every subsequent API request.
11. User lands on the dashboard, scoped to their role.
12. Admin creates or assigns tasks; documents are uploaded and queued for embedding.
13. Uploaded text is converted into vector embeddings and indexed in FAISS.
14. User submits a search query, which is embedded and matched against the FAISS index.
15. The most relevant documents are returned, and the query is logged.
16. User acts on the retrieved knowledge and updates task status accordingly.

10. Security Considerations

- Passwords are hashed (never stored in plaintext) and verified on login.
- JWTs are signed and time-bound, with role claims validated server-side on every protected request.

- RBAC is enforced in the API layer, not just hidden in the UI — a User token cannot reach Admin-only routes regardless of frontend behaviour.
- Input validation is handled through Pydantic schemas at the API boundary before data reaches business logic.

11. Challenges & Design Decisions

- Balancing MVP scope against the assignment's 2-3 day window meant prioritizing the mandatory AI and RBAC requirements over nice-to-have polish.
- Choosing FAISS over calling an external embedding/search API directly addresses the requirement to implement core AI logic rather than depend entirely on a third party.
- Keeping the API layer thin and pushing logic into services makes the codebase easier to extend — for example, swapping FAISS for another vector store later touches one module, not the whole backend.

12. Testing & Verification

- Manual verification of every endpoint via FastAPI's auto-generated Swagger UI (/docs).
- Role-based access spot-checked by calling Admin-only routes with a User token and confirming rejection.
- Search quality checked by querying with paraphrased text (not exact keywords) and confirming semantically relevant documents are still returned.
- End-to-end flow walked manually: login → upload → search → task update → analytics, confirming activity logs populate correctly at each step.

13. Future Improvements

- Support PDF and DOCX ingestion alongside .txt.
- Add an automated test suite (pytest for the backend, React Testing Library for the frontend) and wire it into CI.
- Introduce pagination and response caching on high-traffic endpoints.
- Move from a local FAISS index to a managed vector store (e.g. Chroma or pgvector) to support multi-node deployments.
- Add refresh tokens and rate limiting to harden the authentication layer further.

14. Conclusion

This project delivers a working, end-to-end system that satisfies every mandatory requirement of the assignment: JWT-based authentication with RBAC, a normalized MySQL schema, document ingestion, a self-implemented semantic search pipeline built on Sentence Transformers and FAISS, task management with dynamic filtering, activity logging, and basic analytics. More importantly, it demonstrates a system-design mindset — clear separation between API, business logic, and data layers; a database schema built around real relationships rather than convenience; and an AI feature that owns its core logic instead of proxying it to a third-party API. The result is a foundation that could realistically be extended into a production knowledge-management tool with the improvements outlined above.